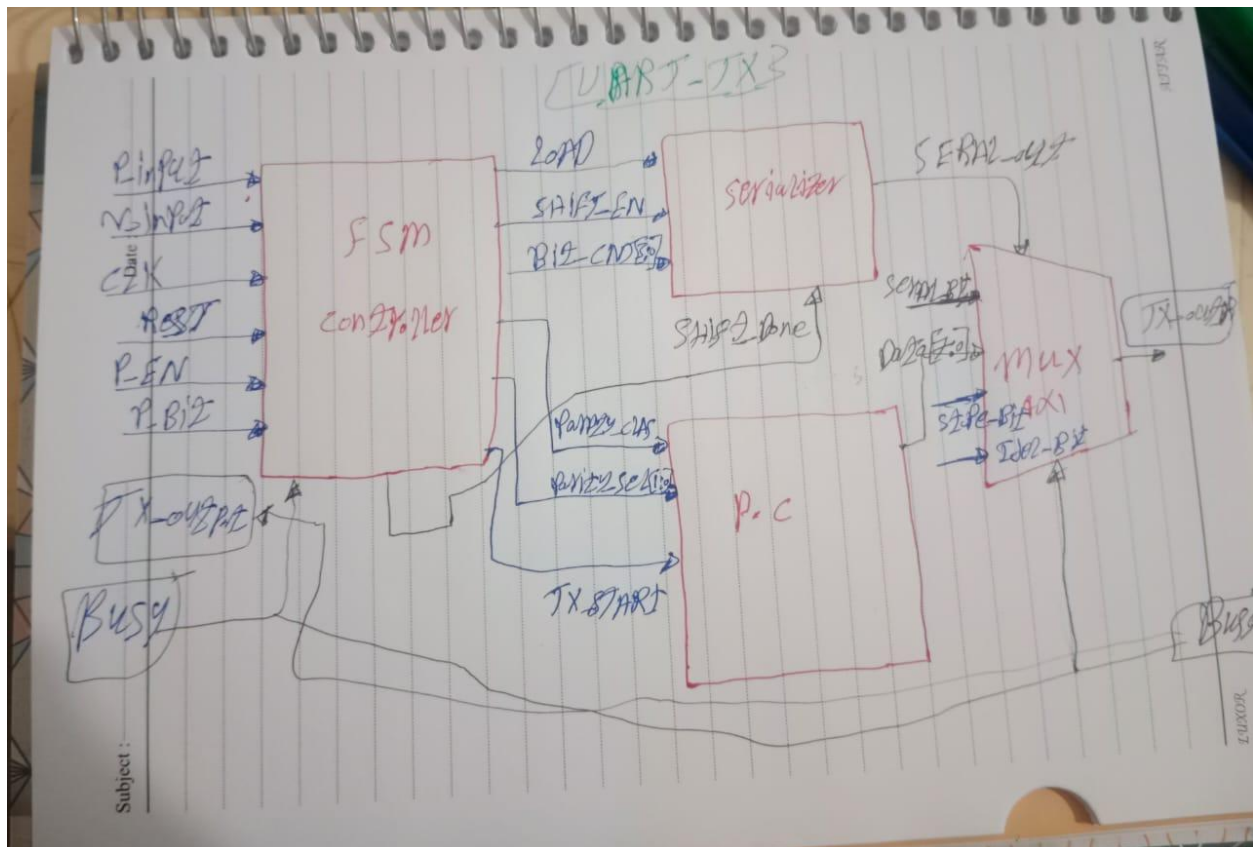


Final Project

UART_TX

الاسم: محمد عبد الخالق عثمان مرسي

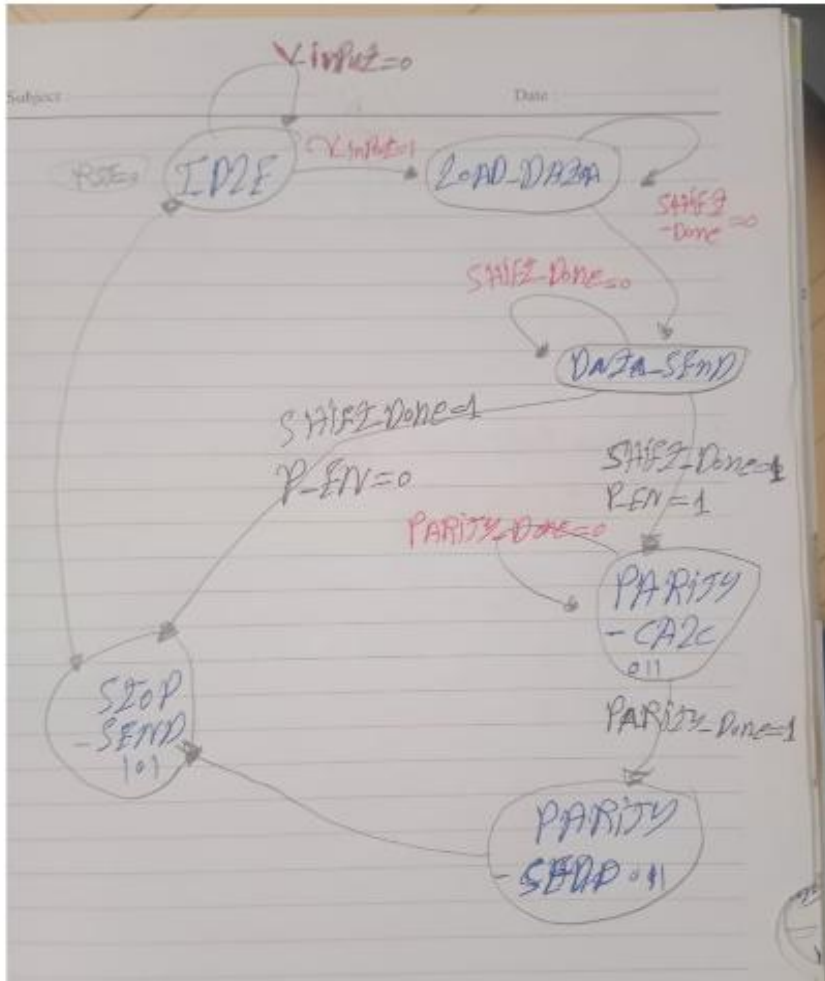
1-Block diagram

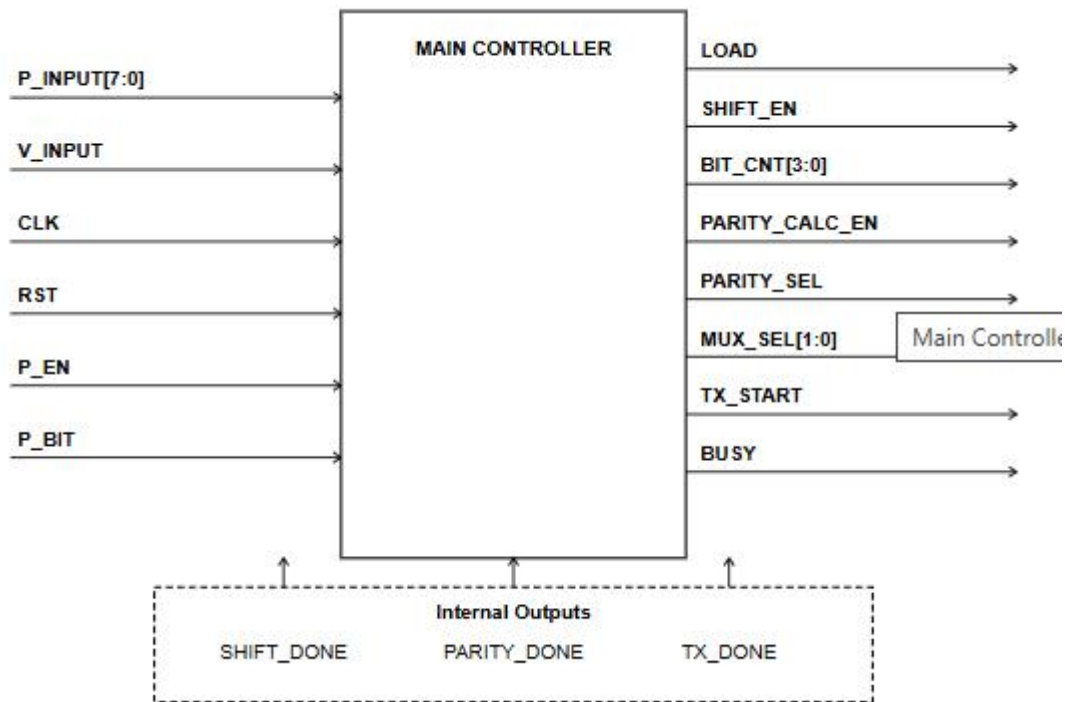


2- MAIN MODULES:

Main Controller fsm

A-fsm desine





ده من - الجزء **Main Controller FSM** : Codeing : B-
 data ال إرسال ترتيب في يتحكم إنه وظيفته **UART TX**.
 stop bit وال parity وال -

next_state يعني الحاله بتتغير عند state اول جزء تخزين
 تتغير clk لما

1. حالة IDLE

دي حالة الانتظار.

بيكون صفر BUSY و مش بيبيت حاجة هنا UART الـ
يبقى واحد V_INPUT يفضل مستني لحد ما Controller الـ

2. LOAD_DATA حالة

P_INPUT اللي موجودة على تحميل للداتا هنا بيحصل

3. DATA_SEND حالة

p_en=1 قدامي طريقين لو shift_done=1 لما
stop_send * p_en = 0 ولو praity_calc هتروح علي

4. PARITY_CALC حالة

PARITY_DONE = 1

:ساعتها ينتقل إلى

PARITY SEND

5. PARITY_SEND حالة

6. STOP_SEND حالة

BUSY = 1

يعني الجهاز مشغول

IDLE: ولما يرجع لـ

BUSY = 0

يعني جاهز يستقبل داتا جديدة

TX هيتحط على خط إيه اللي وظيفته يحدد MUX الـ

- . 00 → Data
- . 01 → Parity
- . 10 → Stop
- . 11 →

c- tp_fsm controller

Main Controller ده معمول لاختبار ال Testbench ال بتنتقل بين الحالات بشكل صحيح أثناء FSM والتأكد إن ال UART TX عملية إرسال البيانات في

، وبعد كده بيتم Controller لل Reset في البداية بيتم عمل تجهيز البيانات وإشارات التحكم. يتم أولاً اختبار إرسال بيانات ، وبعد انتهاء إرسال البيانات ينتقل ال Parity بدون ، ثم يرجع لحالة الانتظار Stop Bit لإرسال ال Controller

في Parity. بعد كده بيتم اختبار إرسال بيانات مع تفعيل ال يحسب Controller الحالة دي بعد انتهاء إرسال البيانات، ال Parity وينتظر إشارة انتهاء الحساب، ثم يرسل ال Parity ال IDLE. ، وأخيراً يرجع لحالة ال Stop Bit وبعدها ال

ويغير إشارات Clock كمان بيولد إشارة ال Testbench ال الدخول في أوقات محددة علشان يحاكي التشغيل الحقيقي و **SHIFT_EN** و **LOAD** ويتأكد إن إشارات الخروج زي بتتغير **BUSY** و **MUX_SEL** و **PARITY_CALC_EN** بالشكل الصحيح.

2-Serializer

والتأكد إنها بتشتغل بشكل صحيح **Serializer** بيستخدم لاختبار وحدة ال **Testbench** الـ

علشان يتم تحميل load ونفعل إشارة p_input للوحدة، وبعد كده بنحط البيانات 11010100 على **Reset** في البداية بنعمل **Serializer** البيانات داخل الـ

واحد وراء الثاني **Serial** بشكل bits علشان يتم إرسال الـ **Clock** 8 لمدة 8 دورات shift بعد التحميل، بنفعل

بعد انتهاء البيانات الأولى، بنكرر نفس العملية مع البيانات 10101010

outputs لمتابعة قيم الإشارات والـ **\$monitor** بنستخدم، أثناء الـ **Simulation** **\$dumpvars** و **\$dumpfile** بينما، **Waveform** بيستخدموا لتسجيل الإشارات وعرضها في الـ

→ **Serial** إخراج → **bits** للـ **Shift** → للبيانات **Load** بيعمل **Serializer** بيتأكد إن الـ **Testbench** باختصار: الـ عند انتهاء الإرسال **Done** إشارة

3-Mux.

Clock = 10 ns وبيكون زمن دورة الـ ns كل 5 CLK

في بداية الاختبار يتم إعطاء القيم الابتدائية لكل الإشارات، مع وضع **V_INPUT** وجعل **P_INPUT** البيانات 10101010 في **PARITY_DONE** و **SHIFT_DONE** و **P_EN** تساوي صفر

لإبلاغ الـ **V_INPUT** ثم يتم تفعيل **RST** بعد ذلك يتم تفعيل

بوجود بيانات جديدة جاهزة للإرسال **Main Controller**

بأن **Controller** لإبلاغ الـ **SHIFT_DONE** بعدها يتم تفعيل **Shift** انتهت عملية الـ

وتحديد P_EN بعد ذلك يتم اختبار إرسال بيانات أخرى مع تفعيل P_INPUT ثم وضع البيانات 11001100 في P_BIT، قيمة V_INPUT. وتفعيل.

PARITY_DONE ثم SHIFT_DONE وفي النهاية يتم تفعيل Parity وحساب ال Shift بانتهاء عملية ال Controller لإبلاغ ال.

4-Parity bit calculator.

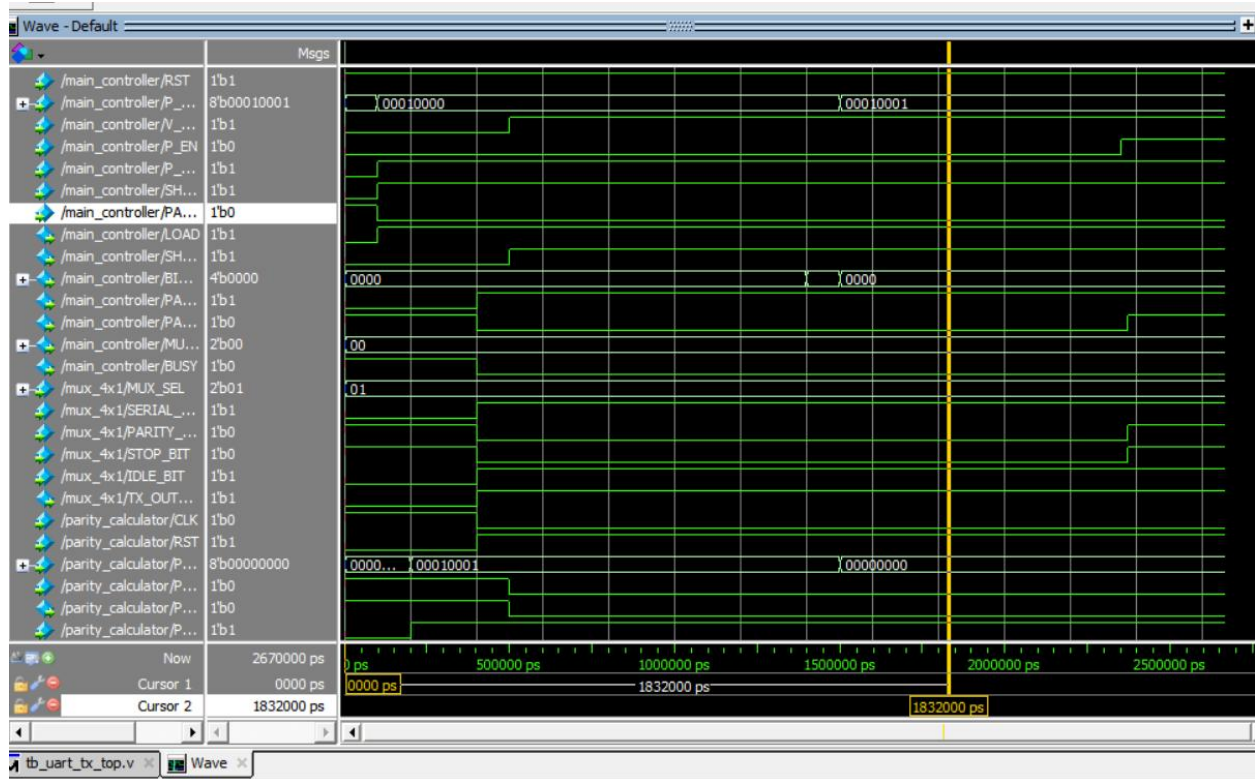
داخل ال always، عند حدوث Clock Edge يتم فحص P_EN. عندما تكون $P_EN = 1$ يتم تخزين نتيجة ال Parity في PARTY_DONE، ويتم تفعيل PARTY_DONE لإعلام ال Main Controller بأن عملية حساب ال Parity انتهت. أما عندما تكون $P_EN = 0$ ، يتم جعل PARTY_DONE = 0.

عند تفعيل RST يتم تصفير كل من PARTY_BIT و PARTY_DONE

5- TOP_Module

الـ Top Module هو الوحدة الرئيسية في تصميم الـ UART Transmitter، ومنها يتم تجميع وربط جميع الوحدات الداخلية مع بعضها. في هذا المشروع الـ Top Module هو uart_tx، ويقوم بربط كل من main_controller و Serializer و parity_calculator و mux_4x1. يستقبل الـ Top Module البيانات وإشارات التحكم، ثم ينظم عملية الإرسال ويحدد الـ bit الذي سيتم إرساله على TX_OUTPUT. كما يوفر الإشارة BUSY لتوضيح حالة الإرسال

WaveForam



WaveForm

- تغيرت من 000000001 ل 00010001 وبعدين P_INPUT
تاني — يبقى فيه تغيير قيم فعلي حصل 00010001
- — اتغيرت من 0000 ل 0001 بعدين رجعت 0000 BIT_CNT
يبقى فيه عد فعلي شغال
- Serializer/p_input 00000000 (بتاخذ نفس القيم كمان,
يبقى فيه توصيل صحيح بين — 00010000, 00010001)
main_controller وال Serializer

